

# Web Technologies



**Web applications: other architectural approaches**  
from **SPA, PWA**, and **mash-up** to **CORS** and **JAMstack**

Prof. Sabin Corneliu Buraga – [profs.info.uaic.ro/sabin.buraga/](https://profs.info.uaic.ro/sabin.buraga/)  
Assoc. Prof. Andrei Panu – [profs.info.uaic.ro/andrei.panu/](https://profs.info.uaic.ro/andrei.panu/)

„The manner of giving is worth more than the gift.”

Pierre Corneille



what does SPA (Single-Page Application) represent



what is PWA (Progressive Web Application)



how can we develop Web *mash-ups*



how to access resources via JavaScript through CORS  
(Cross-Origin Resource Sharing)



what does JAMstack mean

The existence of (micro-)services and REST/GraphQL APIs – eventually, asynchronously invoked via Ajax – offer the premises for developing the following kind of Web apps:

 **SPA** (Single-Page Application)

 **PWA** (Progressive Web Application)

 **hybrid** (Web mash-up)

 **Web game** – e.g., MMORPG  
(Massively Multiplayer Online Role-Playing Game)

# 🚿 SPA (Single-Page Application)



# Web interaction: spa

Dynamic rewriting of Web page content  
– based on asynchronous data transfer –  
following user interaction

advanced

app store

native  
app

Google Drive app  
for Android/iOS

Web browser

SPA  
Single  
Page  
App

[drive.google.com](https://drive.google.com)



HTTP  
Web  
Socket

Web  
applications  
and  
services  
(APIs)

Google Drive API  
Google Drive SDK  
for Java, Node.js, Python

adapted from Adrian Colyer

# Web interaction: spa

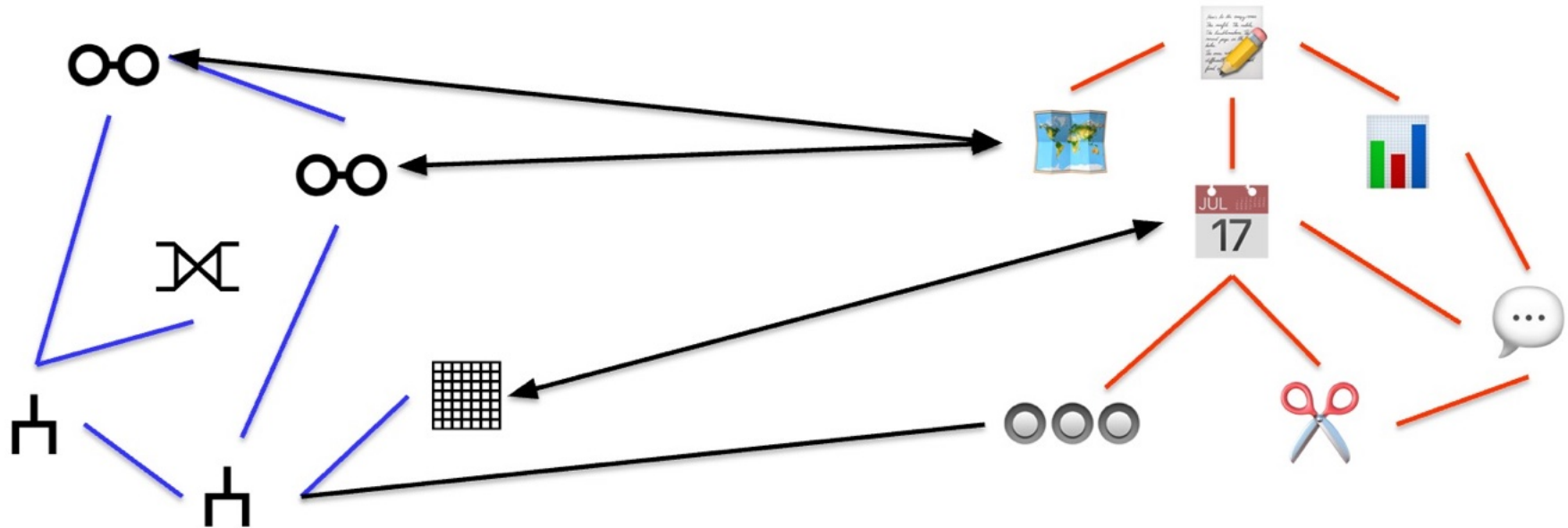
The application will be available at the same base URL, using a single HTML document, the content being rendered in the same Web browser window

in contrast with traditional approaches:  
MPA (Multi-Page Application)



data model

interactive components



at the front-end level, each interactive component (e.g., button, player, edit field, calendar) may be in a certain state (*isPressed*, *isPlaying*, *isTyping*,...)

this state is not necessarily dependent on the back-end

# Web interaction: spa

Problem:

efficient content rendering/refreshing

all the interactive components

- usually, in large number and more complex than those offered by HTML –  
must be synchronized

# Web interaction: spa

DOM alteration operations (mutation) are expensive for complex Web applications

e.g., a collaborative application (Canva, Figma, Google Docs, OverLeaf) or a Web game – for example, see the list  
[www.mmobomb.com/browsergames/mmorpg](http://www.mmobomb.com/browsergames/mmorpg)

# Web interaction: spa

Necessity:

Managing the global application state  
at the Web browser level

# Web interaction: spa

Solution:  
application state management at runtime

URL  $\leftrightarrow$  state

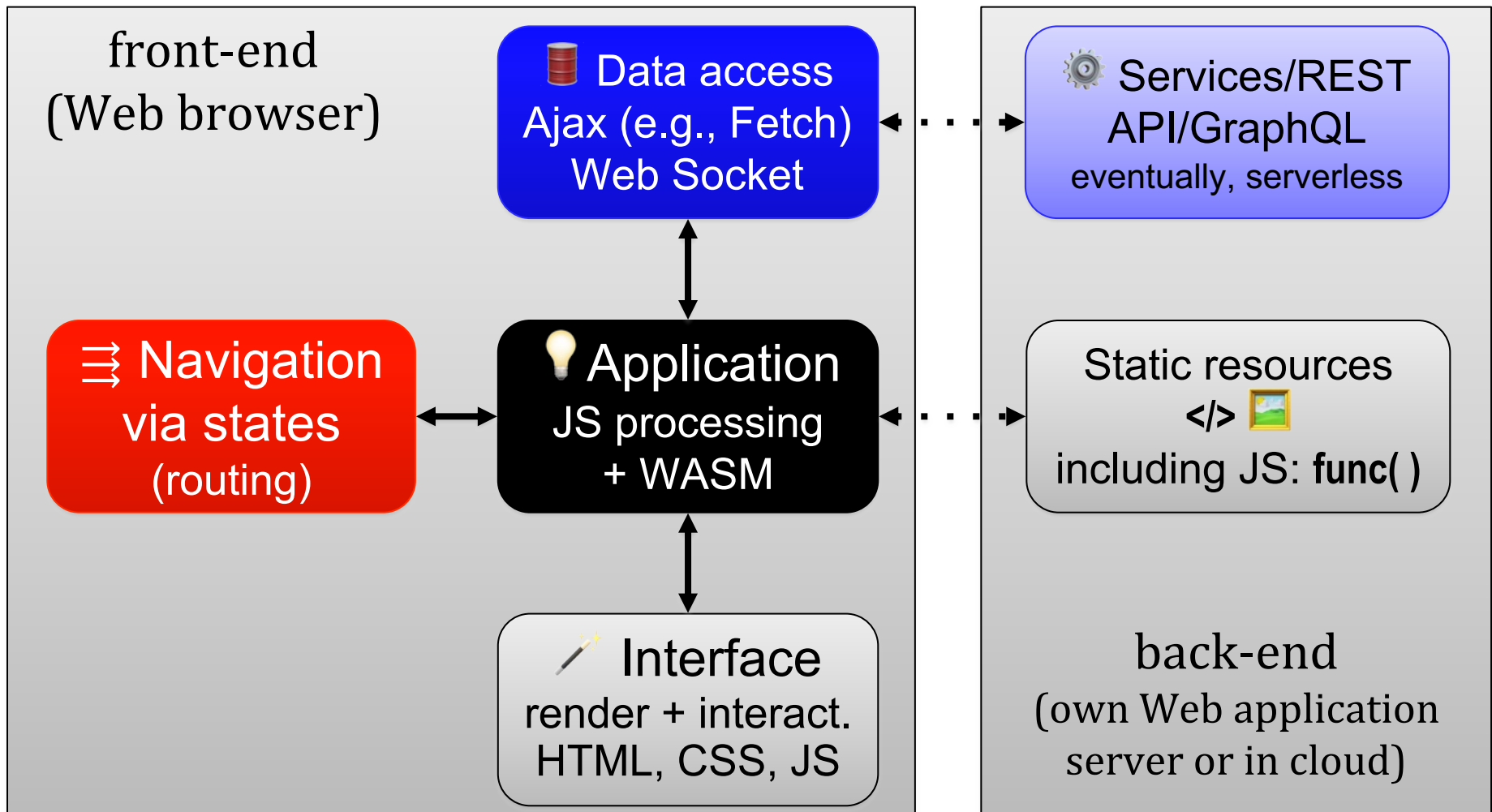
# Web interaction: **spa**

The current state of the view  
is specified by the URL

usually, **/path/state** or **#state**

real example: **Google Drive** – states designated by URLs:  
**/drive/my-drive /drive/recent /drive/shared-with-me /drive/starred**

# Major components of a SPA (adapted from Patrick Ackerman)



# Web interaction: spa

Usually, based on the  
reactive (functional) programming paradigm  
(FRP – functional reactive programming)

targets asynchronously processed data streams  
(event stream, observable)

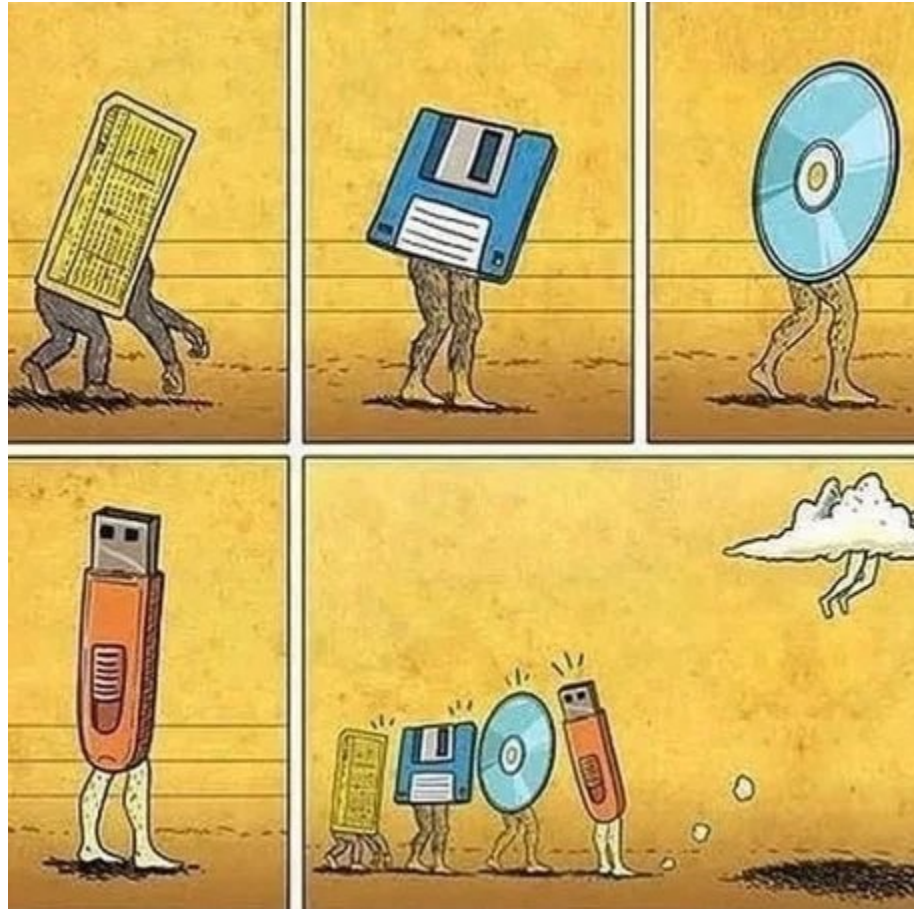
real case studies + resources:

[github.com/stoeffel/awesome-fp-js](https://github.com/stoeffel/awesome-fp-js)

[www.infoq.com/reactive-programming/](http://www.infoq.com/reactive-programming/)



# PWA (Progressive Web Application)



# pwa

Web applications capable of providing functionalities adapted to the execution context available on various platforms at Web browser level

use various APIs provided by the browser

[html-now.github.io](https://html-now.github.io)

[webstatus.dev](https://webstatus.dev)

# pwa

Web applications capable of providing functionalities adapted to the execution context

resources of interest:

[developer.mozilla.org/Web/Progressive\\_web\\_apps](https://developer.mozilla.org/Web/Progressive_web_apps)

[web.dev/explore/progressive-web-apps](https://web.dev/explore/progressive-web-apps)

[www.pwastats.com](https://www.pwastats.com)

# pwa – characteristics

Adaptation to the device

review the discussions on  
responsive (Web) design

[developer.mozilla.org/Learn\\_web\\_development/Core/CSS\\_layout/Responsive\\_Design](https://developer.mozilla.org/Learn_web_development/Core/CSS_layout/Responsive_Design)

# pwa – characteristics

Interactions similar to native applications that are executed on desktop/mobile devices

# pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:

**Web Workers** – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

[developer.mozilla.org/Web/API/Web\\_Workers\\_API/Using\\_web\\_workers](https://developer.mozilla.org/Web/API/Web_Workers_API/Using_web_workers)

# pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:

**Web Workers** – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

the execution environment of a worker is completely separated, the code being run concurrently/in parallel, asynchronously

# pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:

**Web Workers** – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

the resources provided by external services can be asynchronously loaded via HTTPS by JS programs of type **service workers**

[www.w3.org/TR/service-workers/](http://www.w3.org/TR/service-workers/)



## pwa – characteristics

Offer support for caching: offline vs. online

“Service workers essentially act as **proxy servers** that sit between Web applications, and the browser and the network (when available). They are intended to (amongst other things) **enable the creation of effective offline experiences**, intercepting network requests, and taking appropriate action based on whether the network is available, and updated assets reside on the server.”

[developer.mozilla.org/Web/API/Service\\_Worker\\_API](https://developer.mozilla.org/Web/API/Service_Worker_API)

## pwa – characteristics

Data will be transferred “securely” (with HTTPS)  
from a “trusted” source

# pwa – characteristics

The content can be permanently refreshed,  
retrieved via Web (micro-)services

# pwa – characteristics

- The PWA app can be installed
- e.g., eventually, using an app store –  
and shared



# Web mash-ups



# **mash-ups**

## **Hybrid Web applications – mash-ups**

Combination – at client-level and/or server-level –  
of content from multiple data sources,  
retrieved (a)synchronously,  
providing a new functionality/experience

# mash-ups

Example:

we want to create an application that provides music information depending on the user's physical activity, based on some public Web services

# mash-ups

[www.last.fm/api/rest](http://www.last.fm/api/rest)



+



[dev.fitbit.com/build/](http://dev.fitbit.com/build/)

LaFi  
hybrid Web application





## API Guides

- Introduction
- Authentication
- Web How-To
- Mobile How-To
- Desktop How-To
- Auth Spec
- Scrobbling 2.0 Documentation
- Radio API
- Playlists API
- API Tools
- REST Requests**
- XML-RPC
- Terms of Service

## API Methods

- album
- album.addTags
- album.getInfo
- album.getTags
- album.getTopTags
- album.removeTag
- album.search

### API ROOT

The API root URL is located at <http://ws.audioscrobbler.com/2.0/>

Generally speaking, you will send a method parameter expressed as 'package.method' along with method specific arguments to the root URL. The following parameters are required for all calls:

**api\_key** : A Last.fm API Key.

**method** : An API method expressed as *package.method*, corresponding to a documented last.fm API method name.

For example:

```
http://ws.audioscrobbler.com/2.0/?method=artist.getSimilar&api_key=xxx...
```

If you are accessing a *write* service, you will need to submit your request as an HTTP POST request. All POST requests should be made to the root url:

```
http://ws.audioscrobbler.com/2.0/
```

With all parameters (including the 'method') sent in the POST body. In order to perform write requests you will need to authenticate a user with the API. See [authentication](#) for more.

# access to Last.fm services via an authentication key

# mash-ups

## Reference

Device API

Companion API

Settings API

## Web API

Developer Guide

Active Zone Minutes Time Series

Activity

Activity Time Series

Authorization

Body

Body Time Series

## Breathing Rate

Get Breathing Rate Summary by Date

Get Breathing Rate Summary by Interval

## Get Breathing Rate Summary by Interval

This endpoint returns average breathing rate data for a date range. Breathing Rate data applies specifically to a user's "main sleep," which is the longest single period of time during which they were asleep on a given date.

The single date measurements are provided at the end of a period of sleep. The data returned can and often does reflect a sleep period that began the day before. For example, if you request a daily breathing rate for 2021-12-22, it may include measurements that were taken the previous night on 2021-12-21 when the user started to sleep.

It uses [units](#) that correspond to the Accept-Language header provided.

Scope: *respiratory\_rate*

## Request

GET /1/user/[user-id]/br/date/[start-date]/[end-date].json

### URI Arguments

|            |          |                                                                        |
|------------|----------|------------------------------------------------------------------------|
| user-id    | required | The encoded ID of the user. Use "-" (dash) for current logged-in user. |
| start-date | required | The date in the format yyyy-MM-dd or today.                            |
| end-date   | required | The date in the format yyyy-MM-dd or today.<br>Maximum range: 30 days  |

FitBit's REST API provides data in JSON format

(instead of) break



# case study: HuDo

Creating a client-level Web mash-up  
HuDo (Humans 'n' Dogs)

dog images via [dog.ceo/dog-api/](http://dog.ceo/dog-api/) – JSON response:  
`{ "status": "success", "message": "image_URL" }`  
+  
random data about users: [randomuser.me/api/](http://randomuser.me/api/)  
complex JSON response

▼ results:

▼ 0:

gender: "female"

▼ name:

title: "Mrs"

first: "Léane"

last: "Faure"

▶ location: {...}

email: "leane.faure@example.com"

▶ login: {...}

▶ dob: {...}

▶ registered: {...}

phone: "04-74-14-92-94"

cell: "06-28-27-46-95"

▶ id: {...}

▼ picture:

large: "https://randomuser.me/api/portraits/women/35.jpg"

▼ medium: "https://randomuser.me/api/portraits/med/women/35.jpg"

▼ thumbnail: "https://randomuser.me/api/portraits/thumb/women/35.jpg"

nat: "FR"

randomly generated  
JSON data about a  
user's identity  
**randomuser.me**



## HTML

```
1 <div id="images"><span id="human"></span> vs <span id="dog"></span></div>
2 <div id="user-name"><span id="text"></span></div>
```

## JavaScript

```
1 // verificăm dacă avem succes (codul de stare e 2XX)
2 // pe baza unei promisiuni
3 function status(response) {
4   if (response.status ≥ 200 && response.status < 300) {
5     // cererea poate fi rezolvată
6     return Promise.resolve(response)
7   } else {
8     // cererea a fost rejectată
9     return Promise.reject(new Error(response.statusText))
10  }
11 }
12
13 // generăm în cadrul unui element identificat prin 'id'
14 // elementul <img /> pentru a include URL-ul fotografiei
15 // (parametrul 'url' dat funcției) ce reprezintă un animal
16 function genImg(url, id) {
17   const elem = document.createElement('img');
18   elem.setAttribute('src', url);
19   document.getElementById(id).appendChild(elem);
20 }
21
22 // generăm în cadrul unui element identificat prin 'id'
23 // elementul <div> pentru a include un sir de caractere
24 // (parametrul 'text' dat funcției)
25 function genDiv(text, id) {
26   const elem = document.createElement('div');
27   elem.textContent = text;
28   document.getElementById(id).appendChild(elem);
29 }
```

## CSS

```
1 img {
2   height: 100px;
3 }
4 #user-name {
5   font-family: sans-serif;
6 }
```



use two fetch() calls to get  
JSON data from the two Web  
services (public APIs)

the received URLs are used to  
generate, with DOM, the <img>  
corresponding elements

the complete source code at [jsfiddle.net/busaco/hp2gxc78/](https://jsfiddle.net/busaco/hp2gxc78/)

The screenshot shows a web browser's developer tools interface. At the top, two XHR requests are listed:

- XHR GET `https://dog.ceo/api/breeds/image/random` [HTTP/2 200 OK 116ms]
- XHR GET `https://randomuser.me/api/` [HTTP/2 200 OK 231ms]

The second request is selected, and the 'Response' tab is active. The response is in JSON format, showing a list of results. The first result is expanded, showing the following data:

```
{
  "gender": "male",
  "email": "perry.peters@example.com",
  "phone": "019467 49135",
  "name": {
    "title": "Mr",
    "first": "Perry",
    "last": "Peters"
  },
  "location": {
    "city": "Newry",
    "state": "Mid Glamorgan",
    "country": "United Kingdom",
    "street": {
      "number": 3090,
      "name": "York Road"
    },
    "postcode": "GL55 8XB"
  }
}
```

inspecting HTTP requests initiated by the program  
to get JSON responses from the two REST APIs

The HTML document in which data will be „injected”  
– transferred asynchronously via Fetch API –  
coming from the two Web services:

```
<div>  
  <span id="human">  
    <!-- here the image of a person will be included -->  
  </span>
```

vs

```
<span id="dog">  
  <!-- and here, the image of a dog -->  
</span>
```

```
</div>
```

```
<div id="user-name">  
  <span id="text">  
    <!-- display the person's identity -->  
  </span>
```

```
</div>
```



// helper functions which modify the document's DOM tree

```
// generate inside an element identified by 'id'  
// the <img /> element to include the URL of the photo  
// (the 'url' parameter passed to the function) which represents an animal  
function genImg (url, id) {  
  const elem = document.createElement ('img');  
  elem.setAttribute ('src', url);  
  document.getElementById (id).appendChild (elem);  
}
```

```
// generate inside an element identified by 'id'  
// the <div> element to include a string  
// (the 'text' parameter passed to the function)  
function genDiv (text, id) {  
  const elem = document.createElement ('div');  
  elem.textContent = text;  
  document.getElementById (id).appendChild (elem);  
}
```

```
// check if successful (status code is 2XX)
function status(response) {
  if (response.status >= 200 && response.status < 300) {
    // the request can be resolved
    return Promise.resolve(response)
  } else {
    // the request was rejected
    return Promise.reject(new Error(response.statusText))
  }
}
```



check again the lecture  
about Ajax

```
const URLDOGS = 'https://dog.ceo/api/breeds/image/random';

// use Dog API to obtain a random image of a dog
fetch (URLDOGS)
  .then (status)    // was the data received successfully?
  .then (response => response.json ())
  .then (json => {
    // process received JSON data...
    // the 'message' property stores the URL of the dog's image
    genImg (json.message, 'dog');
  })
  .catch (error => { // display the occurred error
    document.getElementById ('dog').textContent = error;
  });
```

```
const URLUSER = 'https://randomuser.me/api';

// obtain data about a fictive user algorithmically generated
fetch (URLUSER)
  .then (status)      // was the data received successfully?
  .then (response => response.json ())
  .then (json => {
    // process received JSON data...
    // get the URL of the fictive person's image
    genImg (json.results[0].picture.medium, 'human');
    // obtain the identity
    genDiv (json.results[0].name.title + ' ' +
            json.results[0].name.first + ' ' + json.results[0].name.last, 'text');
  })
  .catch (error => { // display the occurred error
    document.getElementById ('human').textContent = error;
  });
```

| Status | Met... | Domain                                                                                           | File               | Initiator             | Type | Transferred |
|--------|--------|--------------------------------------------------------------------------------------------------|--------------------|-----------------------|------|-------------|
| 200    | GET    |  dog.ceo        | random             | index.html:47 (fetch) | json | 891 B       |
| 200    | GET    |  randomuser.me  | /api/              | index.html:61 (fetch) | json | 1.43 kB     |
| 200    | GET    |  images.dog.ceo | n02113186_7353.jpg | index.html:29 (img)   | jpeg | 23.99 kB    |
| 200    | GET    |  randomuser.me  | 44.jpg             | index.html:29 (img)   | jpeg | 4.51 kB     |

asynchronous data transfers (JSON + images)

| Status | Met... | Domain         | File               | Initiator             | Type | Transferred |
|--------|--------|----------------|--------------------|-----------------------|------|-------------|
| 200    | GET    | dog.ceo        | random             | index.html:47 (fetch) | json | 891 B       |
| 200    | GET    | randomuser.me  | /api/              | index.html:61 (fetch) | json | 1.43 kB     |
| 200    | GET    | images.dog.ceo | n02113186_7353.jpg | index.html:29 (img)   | jpeg | 23.99 kB    |
| 200    | GET    | randomuser.me  | 44.jpg             | index.html:29 (img)   | jpeg | 4.51 kB     |

|                                                                                                         |
|---------------------------------------------------------------------------------------------------------|
| JSON                                                                                                    |
| <pre>message: "https://images.dog.ceo/breeds/corgi-cardigan/n02113186_7353.jpg" status: "success"</pre> |

asynchronous data transfers (JSON + images)

| Status | Met... | Domain         | File               | Initiator             | Type | Transferred |
|--------|--------|----------------|--------------------|-----------------------|------|-------------|
| 200    | GET    | dog.ceo        | random             | index.html:47 (fetch) | json | 891 B       |
| 200    | GET    | randomuser.me  | /api/              | index.html:61 (fetch) | json | 1.43 kB     |
| 200    | GET    | images.dog.ceo | n02113186_7353.jpg | index.html:29 (img)   | jpeg | 23.99 kB    |
| 200    | GET    | randomuser.me  | 44.jpg             | index.html:29 (img)   | jpeg | 4.51 kB     |

JSON

```

message: "https://images.dog.ceo/breeds/corgi-cardigan/n02113186_7353.jpg"
status: "success"

```

► GET https://images.dog.ceo/breeds/corgi-cardigan/n02113186\_7353.jpg

|             |                          |                                                                                                                                                                               |
|-------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status      | 200 OK ?                 |  <p> Name: n02113186_7353.jpg<br/> Dimensions: 500 × 375<br/> MIME Type: image/jpeg </p> |
| Version     | HTTP/3                   |                                                                                                                                                                               |
| Transferred | 23.99 kB (23.05 kB size) |                                                                                                                                                                               |

asynchronous data transfers (JSON + images)

# case study: DoCa

Similarly, a Web mash-up at client level  
DoCa (*Dogs & Cats*)

dog images (dalmatians) via [dog.ceo/dog-api/](http://dog.ceo/dog-api/)

+

pictures – here, animated GIFs – with cats:

*Cat as a service* (CATAAS) – [cataas.com/doc.html](http://cataas.com/doc.html)

also study the OpenAPI specification: [cataas.com/doc.json](http://cataas.com/doc.json)



## HTML

```
1 <div><span id="dog"></span> vs <span id="cat"></span></div>
2
```

## JavaScript

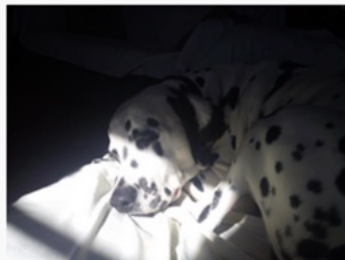
```

31 .then(response => response.json())
32 .then(json => {
33     // procesăm datele JSON primite...
34     // proprietatea 'message' stochează URL-ul
    imaginii câinelui
35     genImg(json.message, 'dog');
36 })
37 .catch(error => { // redăm eroarea survenită
38     document.getElementById('dog').textContent =
    error;
39 });
40
41 // invocăm random.cat API pentru a obține
42 // fotografia unei/unor pisici
43 fetch(URLCATS)
44 .then(status) // datele au fost recepționate cu
    succes?
45 .then(response => response.json())
46 .then(json => {
47     // procesăm datele JSON primite...
48     // proprietatea 'url' stochează URL-ul
    fotografiei cu pisici
49     genImg(json.url, 'cat');
50 })
51 .catch(error => { // redăm eroarea survenită
52     document.getElementById('cat').textContent =
    error;
53 });
54
```

## CSS

```

1 img {
2     width: 200px;
3 }
4
```



vs



| All   | HTML   | CSS           | JS            | XHR         | Fonts     | Images | Media | WS | Oth |
|-------|--------|---------------|---------------|-------------|-----------|--------|-------|----|-----|
| St... | Met... | File          | Initiator     | Transferred | Size      |        |       |    |     |
| 200   | GET    | cooper2.jpg   | img           | cached      | 181,89 kB |        |       |    |     |
| 200   | GET    | gif?json=true | /busaco/z2... | 622 B       | 183 B     |        |       |    |     |
| 200   | GET    | random        | /busaco/z2... | 899 B       | 89 B      |        |       |    |     |
| 200   | GET    | rDKy05t3ROh   | img           | 1,07 MB     | 1,07 MB   |        |       |    |     |

4 requests | 1,25 MB / 1,07 MB transferred | Finish: 1,28 s

Headers Cookies Request Response Timings

Filter properties

JSON

Raw

```

id: "rDKy05t3ROhKsT3P"
tags: (3) [ "kitten", "play", "gif" ]
created_at: "2022-05-01T21:53:01.498Z"
url: "https://cataas.com/cat/rDKy05t3ROhKsT3P?position=center"
mimetype: "image/gif"

```

complete source code at [jsfiddle.net/busaco/z2f3vp4m/](https://jsfiddle.net/busaco/z2f3vp4m/)

# mash-ups

Based on RSS/Atom news feeds,  
Web services, public APIs,...

SaaS (Software As A Service) „current”

implementation at the level of the:  
Web client (browser) and/or Web server

# mash-ups

Characteristics:

combination

visualization

aggregation

# mash-ups

## Combination

use of multiple (heterogeneous) data sources  
can be multidimensional

example:

topic of interest + geographical location + time



a Web mash-up to study the effects of naval pollution  
**EU Shipping Emissions** – [seafloor.pages.dev](https://seafloor.pages.dev)

# mash-ups

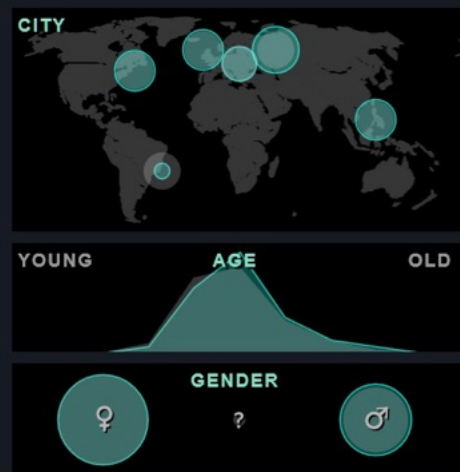
## Aggregation

grouping and analyzing data from multiple sources:  
statistics, classifications, predictions, deductions,...

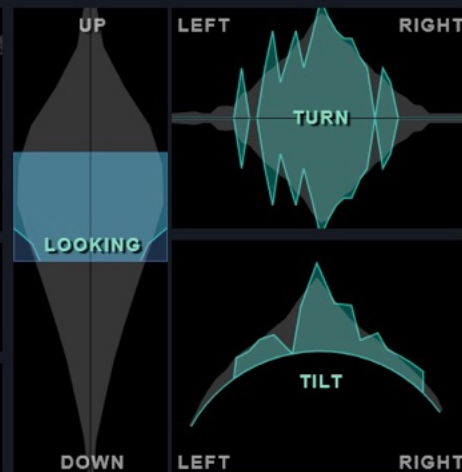
e.g., using machine/deep learning, “hidden” aspects of the  
processed data can be revealed



## DEMOGRAPHICS



## POSE



## FEATURES



## MOOD



70 of 3840 selfies.



classification of images exposed by users  
according to age, posture, mood, and others  
**Selfieexploratory** – [selfiecity.net/selfieexploratory/](https://selfiecity.net/selfieexploratory/)

# mash-ups

## Visualization

Various visualization (presentation) techniques of aggregated data can be adopted:  
charts, cartographic, cloud tags, three-dimensional,...





# Apple Inc

AAPL · Nasdaq Stock Market

## Products

Stocks 45 Futures 1

| Symbol     | Source    |
|------------|-----------|
| AAPL USD   | NASDAQ    |
| APC EUR    | GETTEX    |
| AAPL ARS   | BYMA      |
| APC EUR    | XETR      |
| APC EUR    | TRADEGATE |
| 865985 EUR | LSX       |
| AAPL CAD   | TSX       |
| AAPL MXN   | BMV       |
| AAPL34 BRL | B3        |
| 1AAPL EUR  | MIL       |
| AAPLD USD  | BYMA      |
| AAPL80 THB | SET       |
| AAPL COP   | BVC       |
| 865985 EUR | LS        |
| APC EUR    | FWB       |
| AAPL MXN   | BIVA      |
| AAPLB ARS  | BYMA      |

199.65 USD -1.20 -0.60%

Pre-market. Last update at 15:23 GMT+3

Seasonals Options Bonds

## AAPL chart >



TradingView

Jun Jul

1 day

0.45%

3.71%

-4.04%

-14.46%

Year to date

-19.31%

1 year

5.29%

5 years

151.65%

All time

156.39 K%



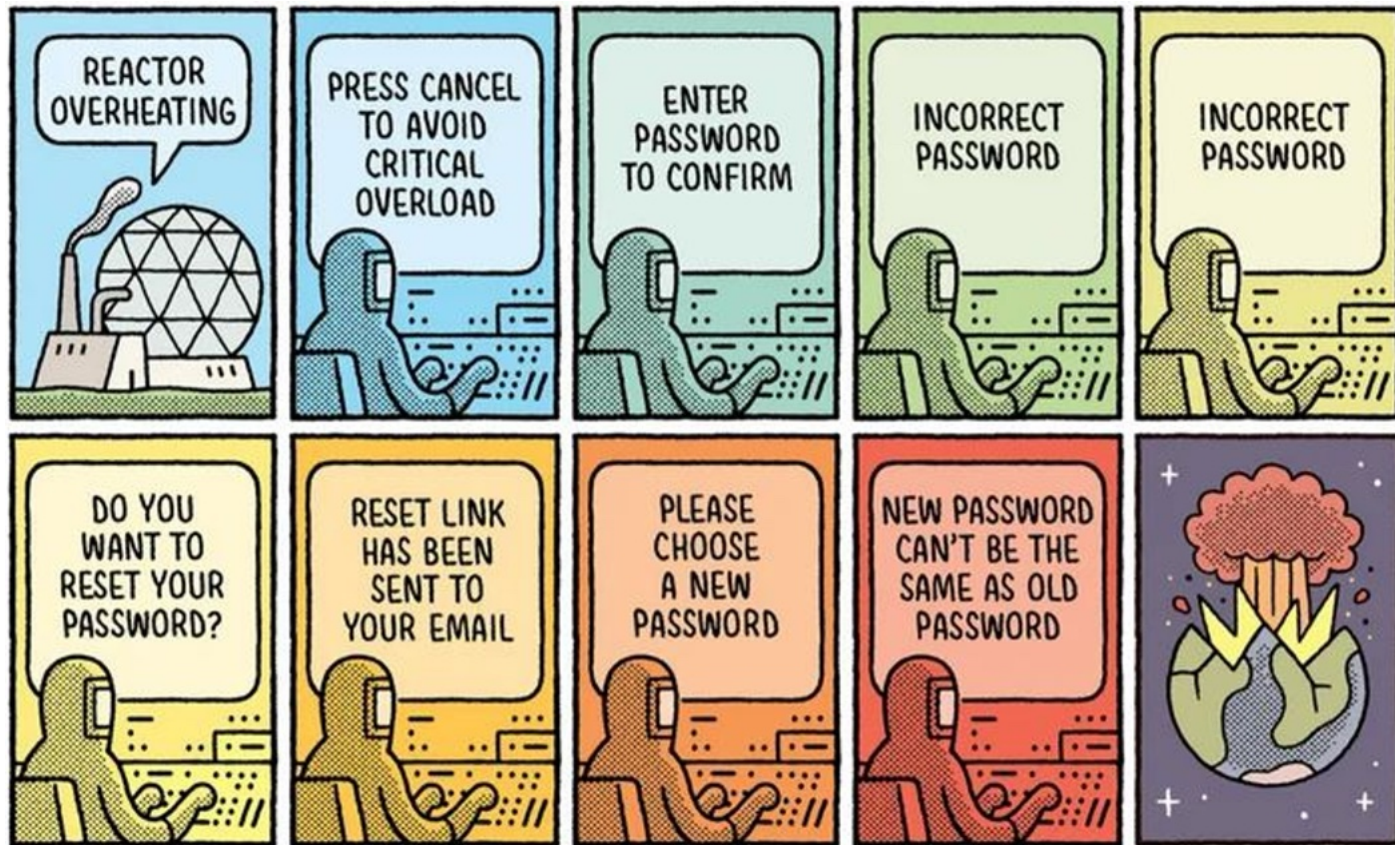
</>

Full chart

analyzing the evolution of a company's shares  
based on multiple data sources – [tradingview.com](https://tradingview.com)

# mash-ups

|                                         |                                                               |
|-----------------------------------------|---------------------------------------------------------------|
| Data sources<br>(data feeds)            | JSON(-LD), RSS, geoRSS,<br>HTML5 microdata, RDFa,...          |
| Programming interfaces<br>(APIs)        | specific to public services and<br>aiming at data processing  |
| Libraries/frameworks<br>for development | generic Web frameworks or<br>offered by organizations         |
| Interactive instruments<br>(Web tools)  | eventually, available in cloud                                |
| Platforms<br>(Platform As A Service)    | Digital Ocean, Heroku, Google<br>Cloud Platform, MS Azure,... |



1/100

TOM GAULD

🧱 Security issues regarding the access to resources, from the browser, via JavaScript?

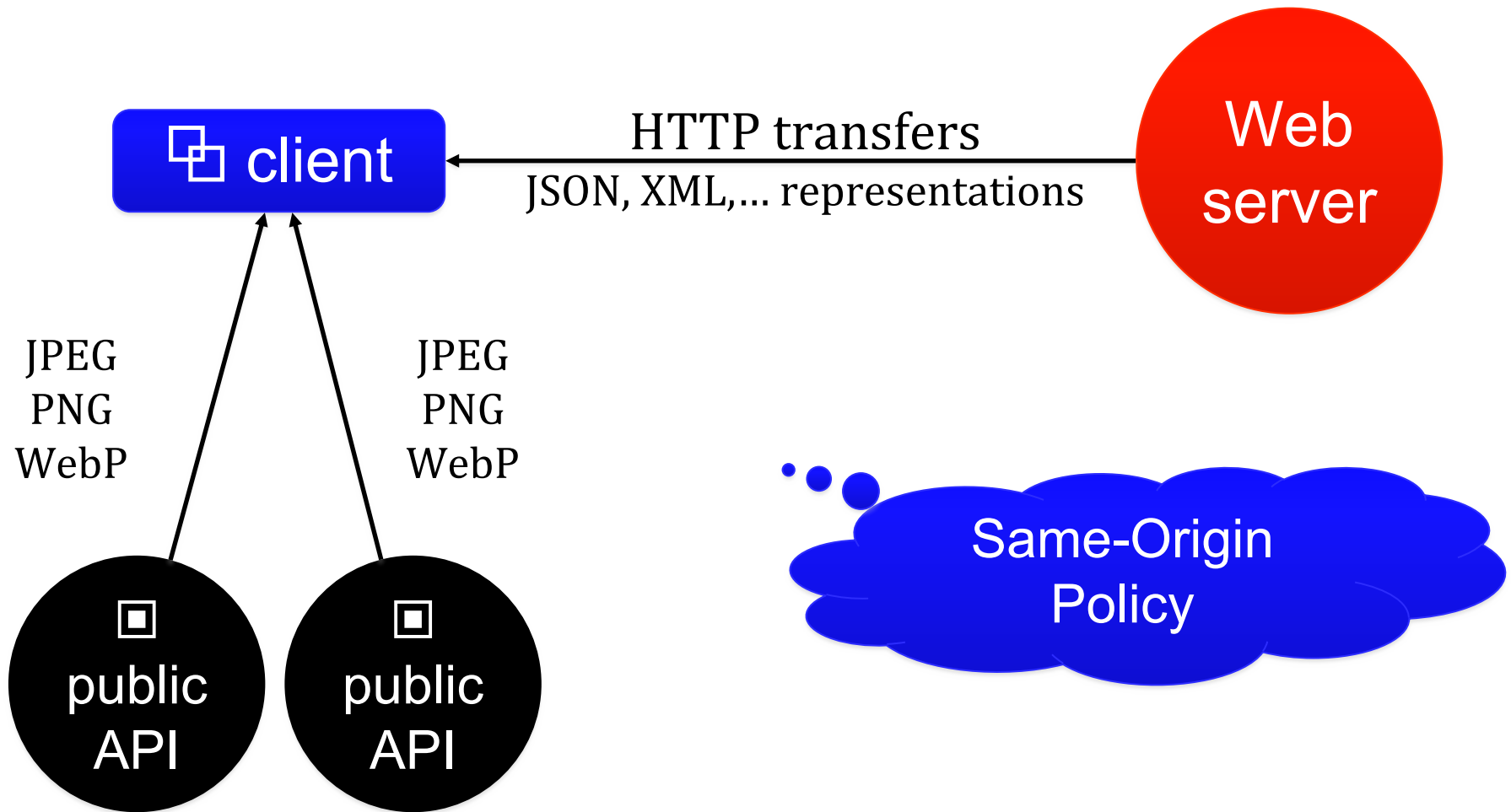
# cors

## Same-Origin Security Policy

“restricts how a document or script loaded from one origin can interact with a resource from another origin”

thus, a JavaScript program must access only data from the same origin – i.e., coming from the same Internet domain

advanced



only transfers involving resource representations regarding images (image/\*), CSS files, and other JavaScript programs from the same origin are allowed

# cors

## Same-Origin Security Policy

prevents the cases when a loaded document/program from an origin can access/modify properties of a document belonging to another origin

[developer.mozilla.org/Web/Security/Same-origin\\_policy](https://developer.mozilla.org/Web/Security/Same-origin_policy)

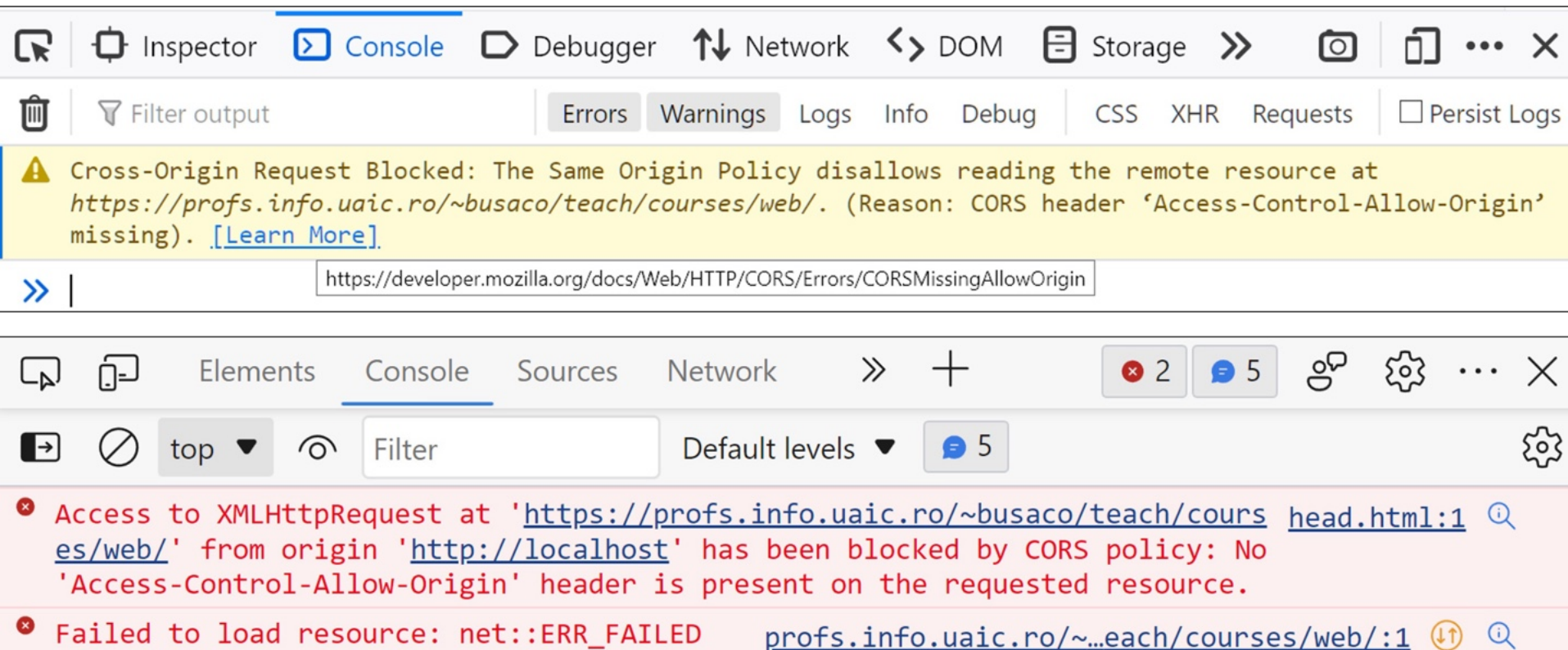
```
const URL = "https://profs.info.uaic.ro/~busaco/teach/courses/web/";

// send a HEAD request to obtain metadata about a resource
let client = new XMLHttpRequest ();
client.open ("HEAD", URL, true);
client.send ();
client.onreadystatechange = function () {
    // did we receive the header fields?
    if (client.readyState == 2) {
        // display the MIME type and the date of the last update
        alert ("The resource of type " +
            client.getResponseHeader ("Content-Type") + " was updated at " +
            client.getResponseHeader ("Last-Modified"));
    }
}
```

asynchronously retrieving,  
with **HEAD**, some metadata



by trying to run in the browser the JS program sent by a Web server installed locally (localhost), we get:



URL of a different domain → the Same Origin Policy is violated



## real case: RDF Converter

▶ XHR GET <https://converter.zazuko.com/> [HTTP/2 200 0ms]

▶  XHR GET <https://profs.info.uaic.ro/sabin.buraga/teach/courses/wade/web-film.html> <https://converter.zazuko.com/> eb-film.h... CORS Missing Allow Origin

! Cross-Origin Request Blocked: The Same Origin Policy disallows reading the remote resource at <https://profs.info.uaic.ro/sabin.buraga/teach/courses/wade/web-film.html>. (Reason: CORS header 'Access-Control-Allow-Origin' missing). Status code: 200. [\[Learn More\]](#)

# cors

## CORS (Cross-Origin Resource Sharing)

allows client-level sharing of resources from different  
**origins** (e.g., Internet domains)

thus, requests can be made between origins (cross-origin)

Fetch Living Standard - CORS protocol (08 May 2026)  
[fetch.spec.whatwg.org/#cors-protocol](https://fetch.spec.whatwg.org/#cors-protocol)

## **cors**

Resource sharing policies are based on certain fields in the HTTP message header

example (only GET requests are allowed, having in the header of the message specific fields from a client belonging to a given origin):

**Access-Control-Request-Method: GET**

**Access-Control-Request-Headers: Content-Type, Accept**

**Origin: <http://localhost:8000>**

# **cors**

Resource sharing policies are based on certain fields in the HTTP message header

resources of interest:

[developer.mozilla.org/en-US/docs/Web/HTTP/CORS](https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS)

also study Steve Hobbs's tutorial (2019):

[auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/](https://auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/)

## **cors: real case #1**

API providing jokes randomly generated

[v2.jokeapi.dev/joke/Any?safe-mode](https://v2.jokeapi.dev/joke/Any?safe-mode)

accepts GET requests sent from any client origin

**Access-Control-Request-Method: GET**

**Access-Control-Allow-Origin: \***

## **cors: real case #2**

Google Play accepts POST requests sent only from the Google Drive service (the origin is <https://drive.google.com>) which include authorization header fields

**Access-Control-Request-Method:** POST

**Origin:** <https://drive.google.com>

**Access-Control-Request-Headers:** authorization, x-goog-authuser



non-standard field

## **cors: real case #3**

Reddit.com allows POST and OPTIONS requests made from any origin, being accepted only certain header fields (access control depends on time)

**Access-Control-Allow-Headers:** Content-Type, Origin, Accept, X-Signature

**Access-Control-Allow-Methods:** POST, OPTIONS

**Access-Control-Allow-Origin:** \*

**Access-Control-Max-age:** 1728000



By using REST APIs/GraphQL,  
couldn't we separate the way data is  
presented from the actual processing?  
What does JAMstack mean?



# JAMstack

Modern approach for developing Web applications  
at the client-level

“A modern Web development architecture based on  
client-side JavaScript, reusable APIs, and prebuilt Markup.”

Mathias Biilmann

vision:

[jamstack.org](https://jamstack.org)

# JAMstack

Modern approach for developing Web applications  
at the client-level

JavaScript

the actual processing is done by the client

(usually, a Web browser) based on the features provided  
by the JS programming language and the related ecosystem

# JAMstack

Modern approach for developing Web applications  
at the client-level

## APIs

the functionalities and data are encapsulated by **reusable APIs** – own or offered by third parties – accessed  
(a)synchronously through HTTPS via JS programs



see previous courses:  
REST, serverless, GraphQL

# JAMstack

Modern approach for developing Web applications  
at the client-level

## Mark-up

content delivery via templates defined by markup languages, pre-processed for presentation to the client (pre-rendering) and stored redundantly by **CDN (Content Distribution Network)**, usually based on specific tools of type **SSG (Static Site Generator)** or **headless CMS**

# JAMstack

 Web client (browser)

content delivery  
via CDN

APIs + (micro-)services



resources + tools:

[github.com/automata/awesome-jamstack](https://github.com/automata/awesome-jamstack)

also study Mathias Biilmann & Phil Hawksworth,  
*Modern Web Development on the Jamstack*, O'Reilly, 2019

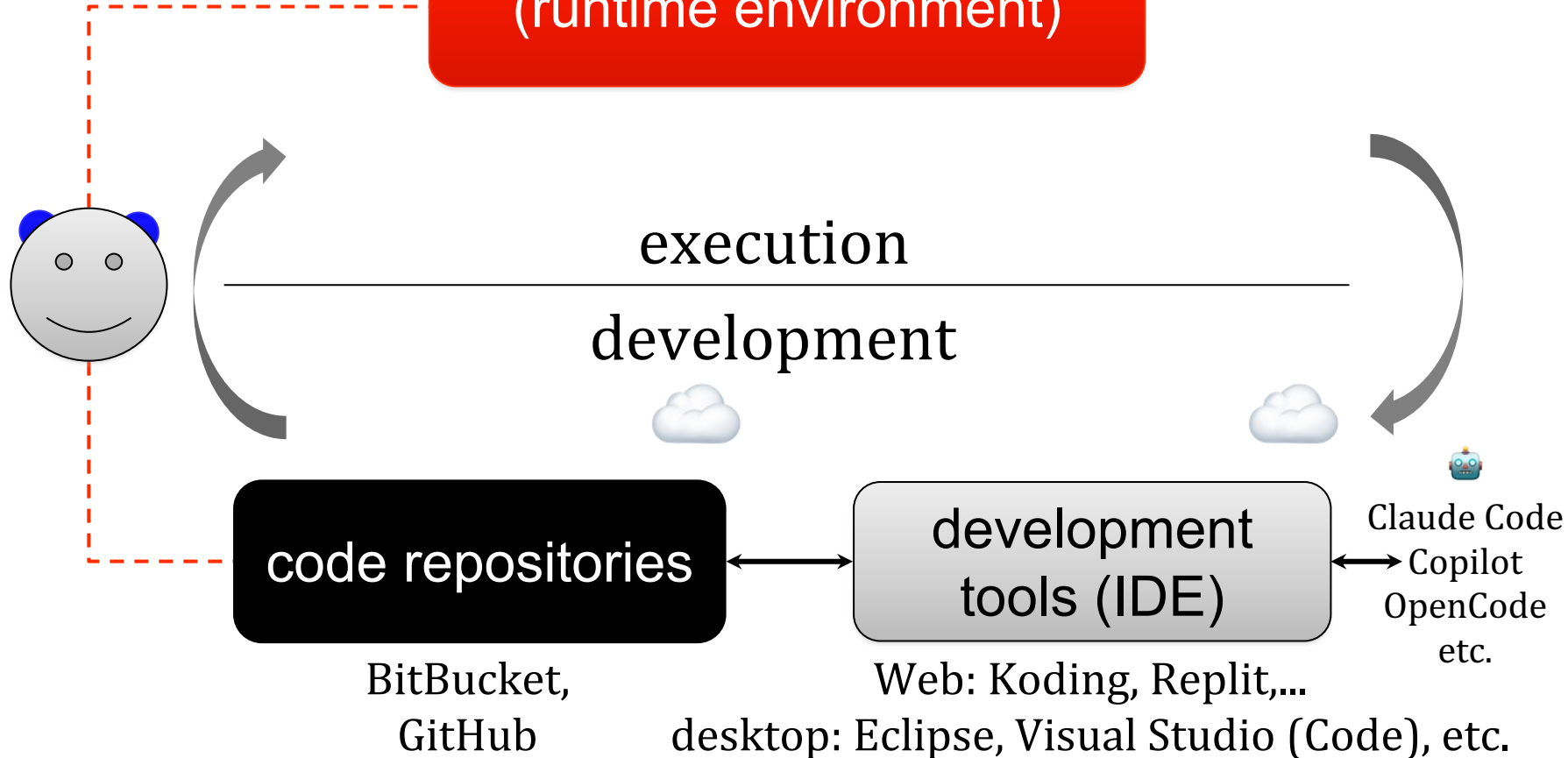
[www.netlify.com/oreilly-jamstack/](http://www.netlify.com/oreilly-jamstack/)

# Development as a Service

DigitalOcean, Google Cloud Platform, Heroku,  
Jelastic, OpenStack, Windows Azure,...

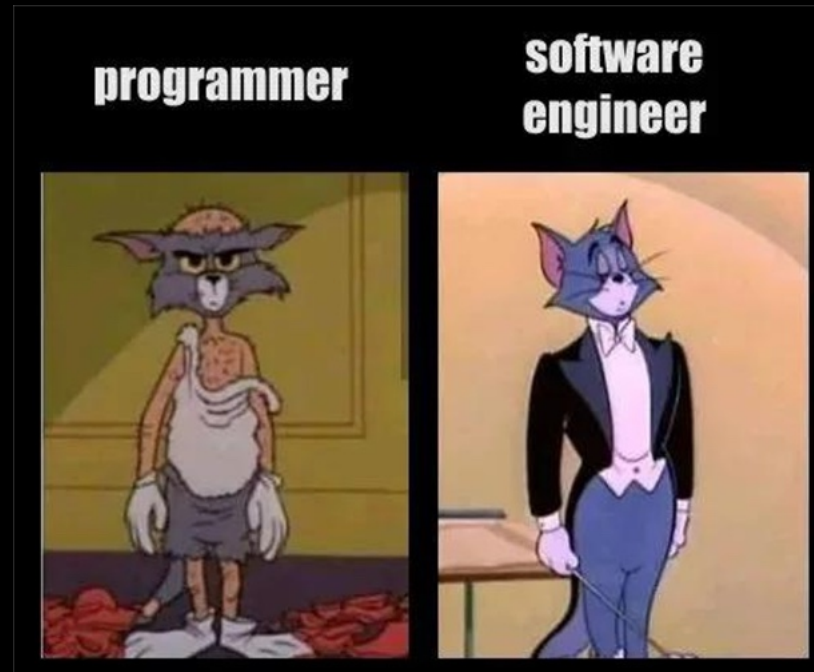


(runtime environment)

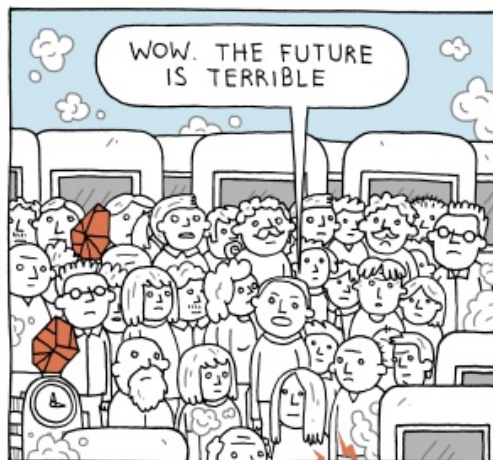
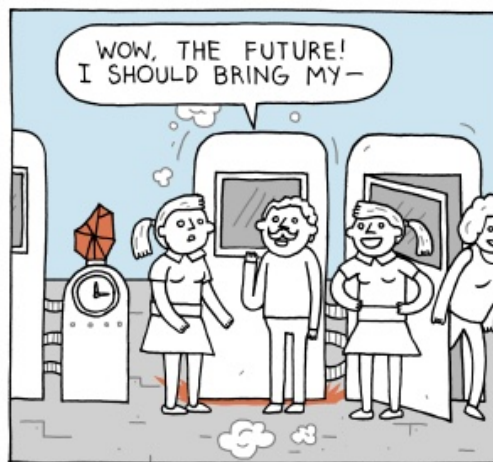
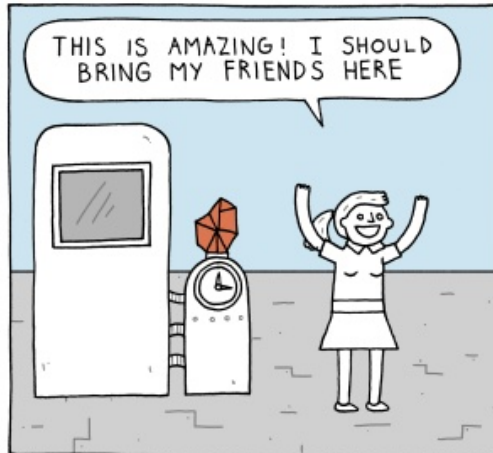
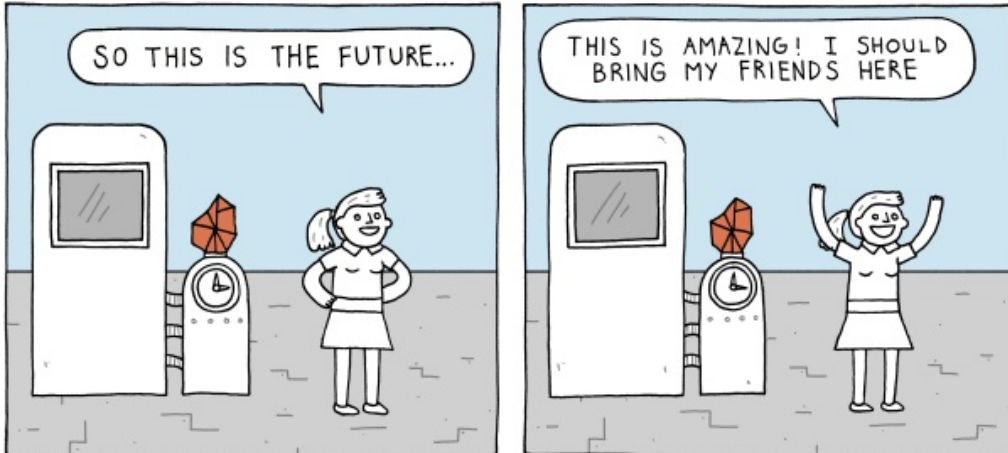


useful tools at [github.com/ripienaar/free-for-dev](https://github.com/ripienaar/free-for-dev)

# summary



**other Web application architectures**  
from SPA and mash-ups to CORS and JAMstack



Good  
luck!